CubeMars AK60-6 V3.0 KV80

Internal notes – MIT protocol, wiring, calibration,
servo-mode vs. MIT-mode, troubleshooting

Repo `exo-actuator-sensor-drivers`

May 12, 2026

Contents

1 What it is

The **CubeMars AK80-9 KV60** is a brushless servo motor with an integrated planetary gearbox and dual magnetic encoders, designed for force-torque controlled robotic joints. It uses the same **MIT mini-cheetah CAN protocol** (hereafter *MIT mode*) that was published by the MIT Biomimetic Robotics Lab for their mini-cheetah actuators and subsequently adopted by many vendors (CubeMars, T-Motor, Damiao, and others).

Property	Value
Model	AK60-6 V3.0 KV80
Voltage	18–52 V (rated 48 V)
Rated current	10 A
Peak current	30 A
Gear ratio	6:1 planetary
Output torque (cont.)	≈5 N·m
Output torque (peak)	15 N·m
Output speed (no-load)	≈45 rad/s at 48 V
Position range	±12.5 rad (firmware MIT limit, `PMAX`)
Encoder	dual 14-bit magnetic (rotor + output shaft)
CAN bus	1 Mbps standard or extended frame
CAN mode (our driver)	**MIT mode only** (see Section 3)
CAN ID (bench)	**0x68 = 104** (confirmed via scan 12 May 2026)
R-Link MIT setting	**Inquiry Feedback** (not "Periodic Feedback")
Configuration tool	R-Link (CubeMars Windows application)

2 What is the "MIT CAN protocol"?

2.1 Background

The MIT mini-cheetah actuator protocol was developed at MIT's Biomimetic Robotics Lab and first described in:

> Ben Katz, *A Low Cost Modular Actuator for Dynamic Robots*, M.Eng. thesis, MIT, 2019.

The protocol sends one 8-byte CAN 2.0A standard frame per motor command cycle. The frame packs five control parameters – position, velocity, stiffness (K_p), damping (K_d), and feed-forward torque – into a single tight bitfield. The motor responds immediately with an 8-byte feedback frame containing position, velocity, and torque.

This is **not** the same as CANopen, UAVCAN/DroneCAN, or any other higher-level protocol; it is a bare, vendor-specific bit-packing scheme over standard CAN 2.0A frames.

2.2 What is "T-Motor CAN"?

T-Motor CAN is the popular name for a close variant of the MIT mini-cheetah protocol used by **T-Motor / CubeMars** products. The name comes from the **T-Motor** brand (same parent company as CubeMars) and is also referred to as:

- *MIT mode* (by CubeMars R-Link and documentation)
- *Mini-Cheetah protocol* (by the robotics community)
- *TMotorCANControl API* (by the open-source `neurobionics/TMotorCANControl` library, which is included as a reference in this repo under `TMotorCANControl/`)

All of the following motors use the same protocol: T-Motor AK series, CubeMars AK series, Damiao DM series (with minor frame differences), and many other MIT-derivative actuators.

2.3 Required physical connection

Layer	Hardware	Notes
Host interface	HDSC USB-to-CAN adapter (USB-ID `2e88:4603`)	Appears as `/dev/ttyACM0` on Linux
Host-to-adapter	USB 2.0 full-speed serial at 921600 bps	The adapter does USB-UART internally
Adapter-to-motor	CAN bus, two wires: **CAN H** and **CAN L**	Twisted pair or shielded cable preferred
Termination	120 Ω resistor between CAN H and CAN L	Required at *both ends* of the bus
CAN bitrate	1 Mbps	Must match R-Link configuration
Power	18–52 V DC, ≥30 A capable supply	Separate from the logic/USB supply

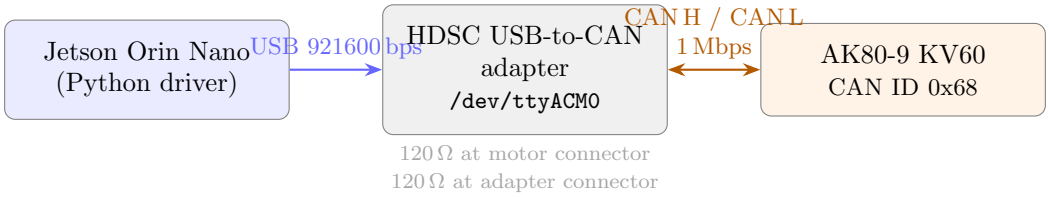

Figure 1: Physical connection chain. The Python driver writes 30-byte HDSC envelope frames to the USB serial port; the adapter converts them to CAN 2.0A standard frames on the CAN bus.

2.4 MIT CAN frame structure

2.4.1 Command frame (host → motor)

One 8-byte payload packed into standard CAN frame with `CAN_ID = ESC_ID`:

Byte(s)	Contents	Bit layout
0	Position upper	`q[15:8]`
1	Position lower	`q[7:0]`
2	Velocity upper	`dq[11:4]`
3	Velocity lower + K_p upper	`dq[3:0] | Kp[11:8]`
4	K_p lower	`Kp[7:0]`
5	K_d upper	`Kd[11:4]`
6	K_d lower + τ upper	`Kd[3:0] | tau[11:8]`
7	τ lower	`tau[7:0]`

Bit-widths: position 16-bit, velocity 12-bit, K_p 12-bit, K_d 12-bit, τ 12-bit. All scaled linearly:

$$\texttt{uint} = \left\lfloor \frac{x + X_{\max}}{2\,X_{\max}} \cdot (2^N - 1) \right\rfloor \quad \text{where } X_{\max} \in \{P_{\max}, V_{\max}, T_{\max}\}$$

For AK60-6: $P_{\max} = 12.5$ rad, $V_{\max} = 45$ rad/s, $T_{\max} = 15$ N·m.

2.4.2 Feedback frame (motor → host)

The motor replies with `CAN_ID = ESC_ID` (not a separate master address as with the Damiao motor):

Byte	Contents	Notes
0	Motor ESC_ID	Redundant with CAN ID
1–2	Position (16-bit)	same scaling as command
3–4	Velocity (upper 8 + lower 4)	12-bit, $\pm V_{\max}$
4–5	Torque (lower 4 + 8)	12-bit, $\pm T_{\max}$
6	MOSFET temperature (°C)	not decoded in current driver
7	Rotor temperature (°C)	not decoded in current driver

2.4.3 Special command frames

Eight-byte payloads with the last byte set to a magic value; the other seven bytes are `0xFF`:

Payload (hex)	Function
`FF FF FF FF FF FF FF FC`	Enable motor (torque ON)
`FF FF FF FF FF FF FF FD`	Disable motor (torque OFF)
`FF FF FF FF FF FF FF FE`	Set current position as zero

3 Two operating modes – the critical distinction

The AK80-9 has **two fundamentally different CAN protocols** that cannot be mixed. The mode is stored in flash by R-Link and takes effect after a power cycle.

	MIT Mode	**Servo Mode ("Periodic Feedback")**
R-Link setting	CAN Mode = **Inquiry Feedback**	CAN Mode = **Periodic Feedback**
CAN frame type	Standard 2.0A (11-bit ID)	Extended 2.0B (29-bit ID)
CAN ID scheme	Command: `ESC_ID` Feedback: `ESC_ID`	Feedback: `(CMD_TYPE ≪ 8) | ESC_ID`
Frame format	8-byte MIT bit-packed	VESC/FESC floating-point extended protocol
Our driver	`cubemars_bus.py` **(supported)**	**Not supported** (extended protocol needed)
TMotorCANControl	`MIT_CAN` class **(supported)**	`servo_can` class
Feedback timing	On request (reply to MIT poll)	Autonomous at configured rate (50 Hz default)

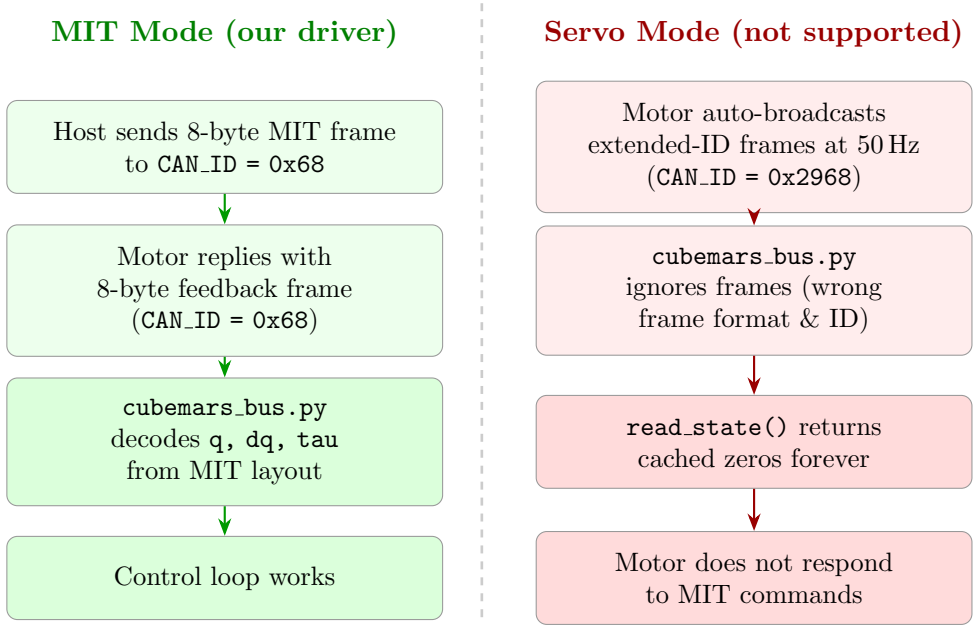

Figure 2: Divergent behaviour depending on the CAN Mode setting in R-Link. The bench motor was found in Servo Mode on 12 May 2026 during initial bring-up; see Section 4 for the diagnostic.

4 Bench bring-up and the CAN-ID discovery problem

4.1 Timeline

During initial bring-up (12 May 2026) the motor was found to be broadcasting extended-CAN frames at all times but not responding to MIT poll commands. Investigation steps:

1. **R-Link screenshot showed:** `CAN Mode = Periodic Feedback`, `CAN ID = ID: 4`. This placed the motor in *Servo mode*, not MIT mode.
2. **MIT poll scan** (`tests/cubemars/00_scan_can_id.py`) sent a null MIT frame (`Kp=Kd=tau=0`) to every CAN ID in `0x01..0x7F`. Result: 1087 frames received, all from extended CAN ID `0x2968`.
3. Decoding the extended ID: `0x2968 = (0x29 << 8) | 0x68` where `0x29` is the VESC/CubeMars servo status command type and `0x68 = 104` is the motor's ESC_ID.
4. **Conclusion:** motor ESC_ID is **104 (0x68)**. The CAN ID shown in R-Link (`ID: 4`) was from a previous *different* configuration page and was stale.

4.2 What the extended CAN ID means

In Servo mode, the motor uses the VESC/FESC extended-CAN protocol:

Field	Value
Full extended CAN ID (29-bit)	`0x2968` = 10600
Command type byte (`cmd = id >> 8`)	`0x29` = 41 = `COMM_GET_VALUES`
Motor ESC_ID byte (`mid = id & 0xFF`)	`0x68` = 104
Broadcast rate	50 Hz (as configured in R-Link)

4.3 Action required

To use `cubemars_bus.py` (MIT mode), open R-Link and apply:

R-Link field	Current value	Required value
CAN Mode	Periodic Feedback	**Inquiry Feedback** (= MIT mode)
CAN Bitrate	1 Mbps	1 Mbps (keep)
CAN ID	4	**104** (or any value – update `motor_config.py`)

Click **Write**, then **power-cycle the motor** (the mode change takes effect on reboot, not immediately).

After switching, rerun the scan:

```
python tests/cubemars/00_scan_can_id.py --start 1 --end 127
```

You should see one MIT feedback frame at the configured ESC_ID (not an extended CAN ID), and the decoder will report valid `q, dq, tau`.

5 Calibration procedure

Unlike the Damiao motor, the AK80-9 has **no firmware parameter read API** accessible over CAN. All calibration steps use R-Link.

5.1 Required steps (one-time after assembly or firmware flash)

1. **Motor Identification** – R-Link spins the motor in both directions to measure winding resistance, inductance, and back-EMF constant. Motor must be unloaded (disconnected from gearbox output if possible, or at least free to spin).
2. **Encoder Identification** – R-Link samples the encoder at many positions and builds a lookup table for pole-pair offset. Must be done immediately after Motor Identification.
3. **Write Parameters** – Click *Write* in R-Link to flash the identified values. Save `.AppParams` and `.McParams` files as backups.

Re-calibrate after:

- Swapping any two of the three motor phase wires.
- Replacing the motor or gearbox.
- Flashing new firmware.

5.2 Software zero position

`cubemars_bus.py` supports software-latching of the current angle as 0 rad via the special frame `[FF FF FF FF FF FF FF FE]`:

```
with open_cubemars_bus(set_zero=True) as bus:
    # motor's current angle is now 0 rad for this session
    bus.write("goal_position", {"j1": 1.0}, kp=30, kd=1.0)
```

This is volatile – it resets on power-cycle.

6 MIT control modes

The five MIT parameters give access to three practical control modes by choosing which are nonzero:

Mode	K_p	K_d	q_{des}	$\dot{q}_{\text{des}}$
Position hold	> 0	> 0	target rad	0
Velocity tracking	0	> 0	0	target rad/s
Open-loop torque	0	0	0	0

Warning: never command $K_p > 0$, $K_d = 0$ in position mode. Without damping the motor oscillates or runs away.

6.1 Recommended gains for AK60-6

Task	K_p	K_d	Notes
Stiff position hold	80–150	2.0–4.0	Heavy load; increase K_d first
Compliant hold	30–60	1.0–2.0	Lighter load or back-drivable
Impedance (teaching)	0–10	0.3–0.8	Very compliant; record demos
Velocity tracking	0	1.0–3.0	$K_p = 0$ mandatory
Torque (open-loop)	0	0	Max 1 N·m unloaded (T_MAX=15 N·m)

6.2 Impedance / kinesthetic teaching

To make the motor back-drivable for demonstration recording:

```
with open_cubemars_bus() as bus:
    while recording:
        q, dq, tau = bus.read_state()["j1"]
        bus.write("goal_position", {"j1": q}, # spring at current pos
                  kp=5.0, kd=0.5) # very soft
        record(t, q, dq, tau)
```

Replay with higher stiffness:

```
bus.write("goal_position", {"j1": target_q}, kp=60, kd=2.0)
```

7 HDSC USB-to-CAN adapter – frame envelope

The Python driver does not use a SocketCAN or PCAN interface; it writes a vendor-specific 30-byte serial envelope directly to the HDSC adapter. The adapter converts this internally to a CAN 2.0A standard frame.

7.0.1 Transmit envelope (30 bytes, host → adapter)

Bytes	Value (hex)	Meaning
0	`55`	Start-of-frame
1	`AA`	Start-of-frame
2	`1E`	Frame length = 30
3	`03`	Command type (send CAN frame)
4–8	`01 00 00 00 0A`	Fixed header fields
9–12	`00 00 00 00`	Reserved
13	`CAN_ID[7:0]`	CAN ID lower byte = ESC_ID
14	`CAN_ID[10:8]`	CAN ID upper bits (for IDs ≤0x7FF always 0)
15–20	`00 00 08 00 00 00`	Fixed (DLC=8)
21–28	payload	8-byte MIT payload
29	(computed)	Unused / padding

7.0.2 Receive frame (16 bytes, adapter → host)

Bytes	Value (hex)	Meaning
0	`AA`	Start byte
1	`11`	Command type (motor feedback)
2	`08`	DLC = 8
3–6	CAN ID (4-byte LE)	For MIT: CAN_ID = ESC_ID
7–14	payload	8-byte MIT feedback payload
15	`55`	End byte

The driver scans for `0xAA` start / `0x55` end markers (16 bytes apart) and discards partial frames.

Servo-mode extended CAN ID. When the motor is in Servo Mode, the feedback CAN ID is 29-bit (`0x2968` for our motor), but the HDSC adapter still delivers it in the 4-byte little-endian field (bytes 3–6). The driver compares against the configured ESC_ID (`0x68`) and these 4 bytes will never match (they decode to `0x00002968` = 10600), so *all servo-mode frames are silently discarded.* This is the root cause of the "all zeros" symptom.

8 Python driver architecture

File	Role
`src/motor_config.py`	Declarative config dataclasses + `DEFAULT_CUBEMARS_BENCH_CONFIG`
`src/cubemars_bus.py`	Self-contained MIT bus driver (`CubemarsMotorsBus`)
`src/_common.py`	`open_cubemars_bus()` context-manager helper
`tests/cubemars/`	Numbered test scripts 00–07

8.1 Context-manager idiom (all tests)

```python
from _common import open_cubemars_bus

with open_cubemars_bus(set_zero=True) as bus:
    bus.write("goal_position", {"j1": 1.57}, kp=30, kd=1.0)
    q, dq, tau = bus.read_state()["j1"]
```

8.2 Test script inventory

Script	Purpose
`00_scan_can_id.py`	Sweep IDs 0x01–0x7F, identify motor; also detects Servo Mode
`01_check_connection.py`	Poll 5 feedback frames, confirm non-zero
`02_mit_position.py`	MIT position hold, step through angles
`03_mit_velocity.py`	MIT velocity tracking
`04_mit_torque.py`	Open-loop torque ($\leq$1 N·m unloaded)
`05_kinesthetic_record.py`	Hand-guide + save `demo_trajectory_ak80.csv`
`06_replay_trajectory.py`	Replay the recorded CSV
`07_impedance_spring_return.py`	Soft spring anchored at zero

9 Known issues and gotchas

1. **[CURRENT ISSUE] Motor is in Servo Mode.** The motor auto-broadcasts extended-CAN status frames at 50 Hz (CAN ID `0x2968`, cmd `0x29`, motor ID `0x68` = 104). `cubemars_bus.py` speaks MIT only (standard CAN). All feedback is silently discarded; `read_state()` returns zeros. **Fix:** R-Link → CAN Mode = **Inquiry Feedback** → Write → power-cycle. feedback frame is 104 (`0x68`). `motor_config.py` has been corrected to `can_id=0x68`.

2. **Model is AK60-6 V3.0 KV80, not AK80-9.** R-Link hardware/software version string `AK60_6V` / `AK60_6_SE_V3` confirmed on 12 May 2026. All MIT limits updated to AK60-6 values: `CUBEMARS_LIMITS["AK60-6"] = [12.5, 45.0, 15.0]`.

3. **Never Kd = 0 in position mode.** Same rule as Damiao MIT mode: $K_d = 0$ with $K_p > 0$ removes all damping and causes oscillation or runaway.

4. **Calibration required before first use.** Run Motor Identification + Encoder Identification in R-Link with the motor unloaded. Write parameters. Re-run if any phase wire is swapped.

5. **No mode switch over CAN.** The MIT / Servo mode switch is a flash-stored setting only changeable via R-Link + power-cycle. It cannot be commanded over CAN at runtime.

6. **Peak current caution.** The motor allows up to 30 A peak. When testing open-loop torque (`04_mit_torque.py`), keep $\tau_{\text{ff}} \leq 1$ N·m when unloaded to avoid damaging a current-limited bench supply.

10 TMotorCANControl reference library

The `TMotorCANControl/` directory contains the neurobionics/TMotorCANControl open-source library, included as a vendored reference. It implements the same MIT CAN protocol for T-Motor / CubeMars actuators using **SocketCAN** (Linux `can0` interface via `python-can`).

10.1 How it differs from our driver

Aspect	`cubemars_bus.py` (ours)	TMotorCANControl
CAN interface	HDSC USB-to-CAN serial (`/dev/ttyACM0`)	SocketCAN (`can0`, `slcan`, etc.)
Dependency	`pyserial` only	`python-can`, SocketCAN kernel module
Jetson setup	Plug-in USB adapter, no kernel config	Requires `ip link set can0 up`
MIT protocol	Identical bit-packing	Identical bit-packing
Servo mode	Not supported	Supported via `servo_can` class
API style	LeRobot-style `read`/`write`	Iterator / context manager

10.2 Using TMotorCANControl (reference, not our primary path)

If you want to switch to SocketCAN on the Jetson (for lower latency or multi-motor daisy-chaining without the HDSC adapter):

```
# Bring up SocketCAN interface (HDSC adapter exposes slcan):
sudo slcand -o -c -s8 /dev/ttyACM0 can0
sudo ip link set can0 up type can bitrate 1000000

# Then use TMotorCANControl:
from TMotorCANControl.mit_can import TMotorManager_mit_can
with TMotorManager_mit_can(motor_type="AK80-9", device="can0",
                           motor_ID=104) as m:
    m.set_zero_position()
    m.update()
    print(m.position, m.velocity, m.torque)
```

Note: the HDSC adapter firmware does not natively speak `slcan`, so the above may require the `slcan` TTY line discipline. Our `cubemars_bus.py` avoids this by writing the HDSC proprietary 30-byte envelope directly, which is more reliable on Jetson without kernel module setup.

11 Quick-start checklist

1. Power on motor (18–52 V; bench uses 48 V).
2. Connect HDSC USB-to-CAN adapter to Jetson USB port.
3. Verify `ls /dev/ttyACM0` exists.
4. **Ensure motor is in MIT mode** (R-Link → CAN Mode = MIT, Write, power-cycle).
5. Run scan to confirm CAN ID:

   ```
   conda activate actuators
   cd tests/cubemars
   python 00_scan_can_id.py
   ```

6. Expected: one MIT feedback frame at `0x68` (104) with valid `q, dq, tau`.
7. If scan shows extended-ID `0x2968`: motor is still in Servo Mode. Return to step 4.
8. Run connection check:

```
python 01_check_connection.py
```

9. Proceed with higher test scripts (02–07).